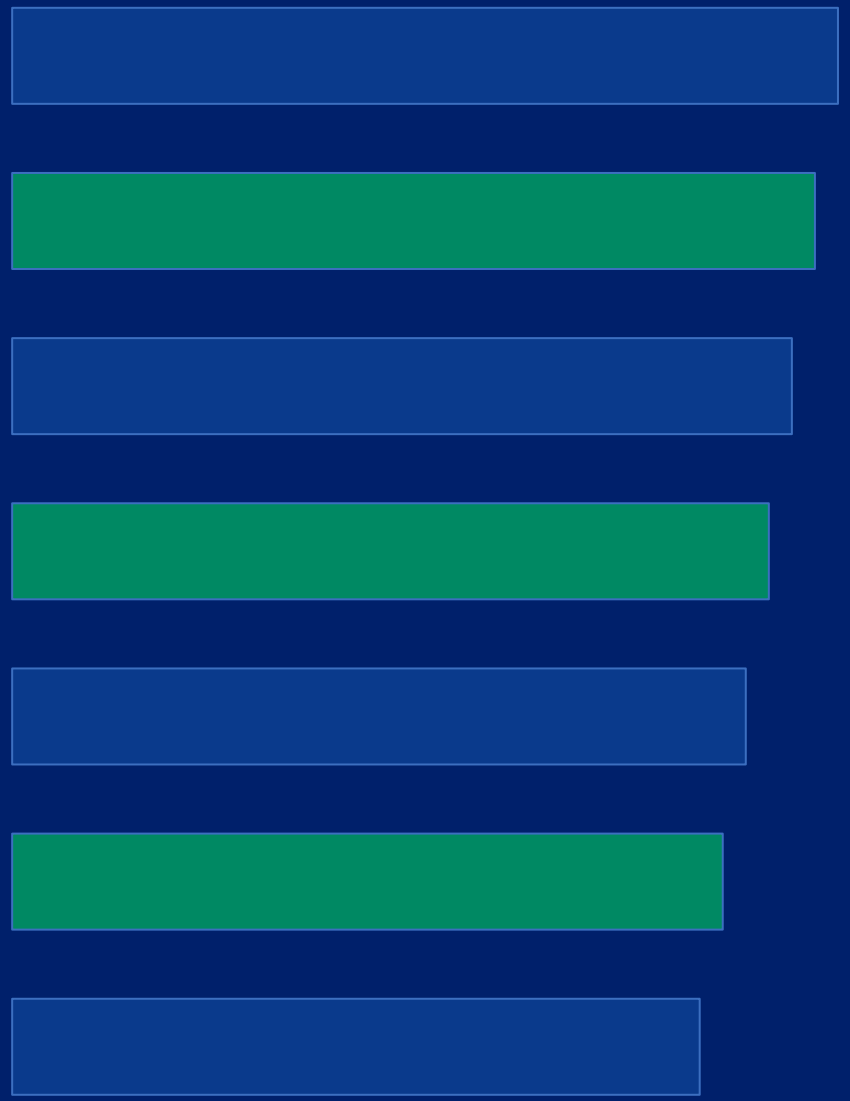
Adopting Agentic Development

A Practical Blueprint for AI-Driven Engineering Teams

KCDC

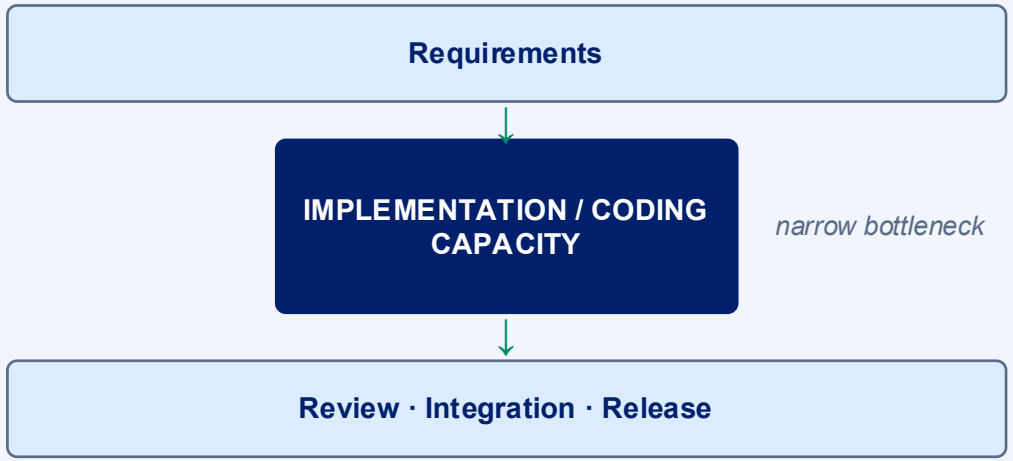


Work System · Context System · Execution System · Guardrails · Integration

PREMISE

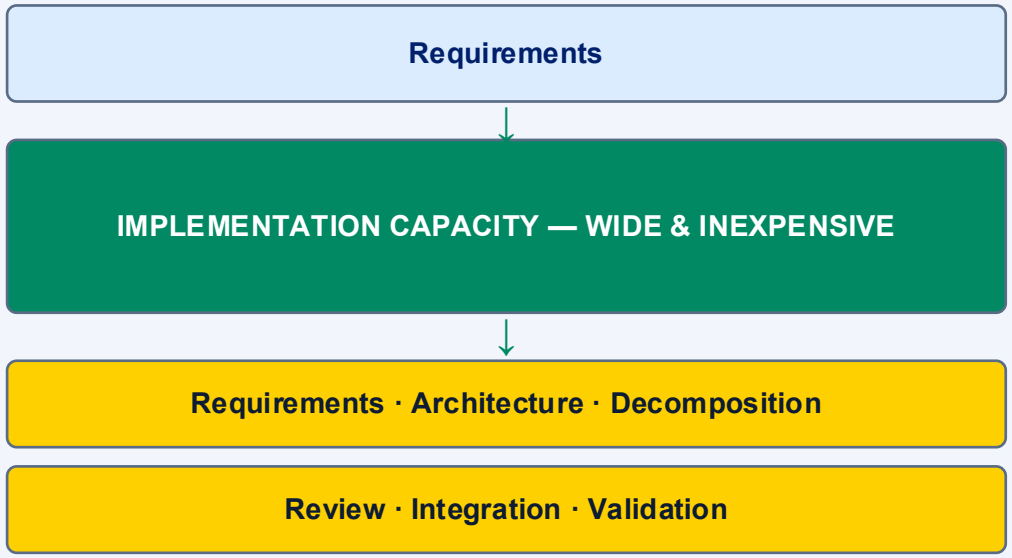
Code Is Becoming Cheap. Engineering Judgment Is Becoming the Constraint.

TRADITIONAL DEVELOPMENT



Optimization target: type faster,
reuse more, hire more implementers.

AGENTIC DEVELOPMENT



New bottlenecks: engineering judgment

More code does not create more delivery throughput. The bottleneck moves — it does not disappear.

DISCIPLINE

Abundant Implementation Lets Software Engineers Spend More Time Engineering

STRUCTURAL ENGINEER

- Defines loads
- Specifies materials
- Designs the structure
- Establishes tolerances
- Defines inspections

Specialists execute:

welder | riveter | fabricator | concrete crew

SOFTWARE ENGINEER

- Defines intent
- Designs architecture
- Decomposes work
- Establishes constraints
- Defines validation

Agents execute:

implementation | test | documentation | dependency | security-analysis

The engineer becomes architect, mentor, and evaluator — not merely the person manufacturing the artifact.

CONTEXT

We Have Been Learning This at Organizational Scale

600+

engineers globally

100+

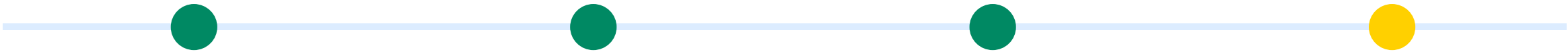
engineers in focus (infrastructure product lines)

~18 mo

of active adoption

Crawl → Sprint

maturity progression in flight



Crawl

most teams today

Walk

many teams today

Run

some teams & workflows

Sprint

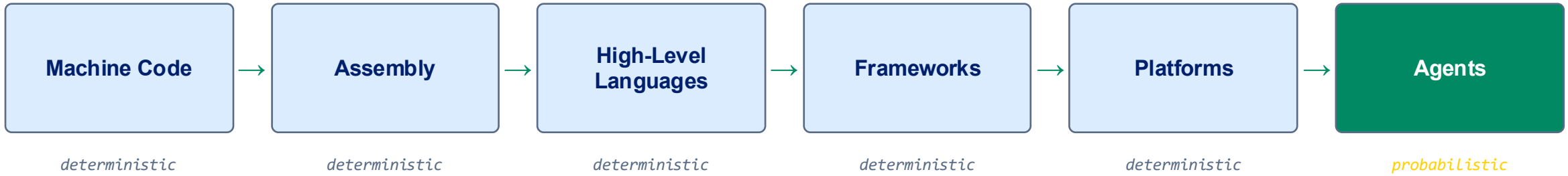
none yet

Some repeatable workflows now show dramatic cycle-time reduction — measured in hours, not sprints.

These lessons come from team and SDLC-level adoption — not individual AI experimentation.

MENTAL MODEL

Stop Treating AI as Magic. Treat It as the Next Abstraction Layer.



As abstraction rises, engineers express more intent and less implementation.

A compiler does not reinterpret your requirement because it found a more convenient solution. An agent can.
That single difference is why the surrounding engineering system matters more than the model.

Agents are the next step in expressing intent — but the first abstraction layer that is probabilistic.

MENTAL MODEL

Treat the Agent Like a Brilliant Junior Consultant on Day One

WHAT IT IS GOOD AT

- Writing code quickly and tirelessly
- Reading APIs and unfamiliar codebases
- Creating tests and refactoring
- Analyzing patterns across many files
- Working in parallel, without fatigue

WHAT IT DOES NOT INHERENTLY KNOW

- Your architecture and service boundaries
- Business context and customer promises
- Supported libraries and licensing policy
- API compatibility guarantees
- Security and operational standards
- Why previous decisions were made

It knows only what you teach it — or what you allow it to discover.

FORCING FUNCTION

Agents Do Not Create Your Process Problems. They Expose Them.

- Tribal Knowledge
- Undocumented Architecture
- Inconsistent Tooling
- Implicit Standards

→

HUMAN ENGINEERS

Compensate silently: ask a colleague, recall history, infer the unwritten rule.

AGENTS

Cannot infer organizational intent — produce fast, plausible, inconsistent outcomes.

→

MAKE IT EXPLICIT

- ADRs & architecture catalogue
- Dependency & licensing policy
- API and compatibility standards
- Testing expectations
- Supportability rules
- Executable conventions (task/make)

If the process only works because “someone knows how we do it,” it is not an engineered process.

OBSERVED EFFECT

We Increased Coding Throughput — and Discovered the Next Bottlenecks

NOW FAST & ABUNDANT

- Coding & refactoring
- Test generation
- Documentation generation
- Boilerplate & migrations

→

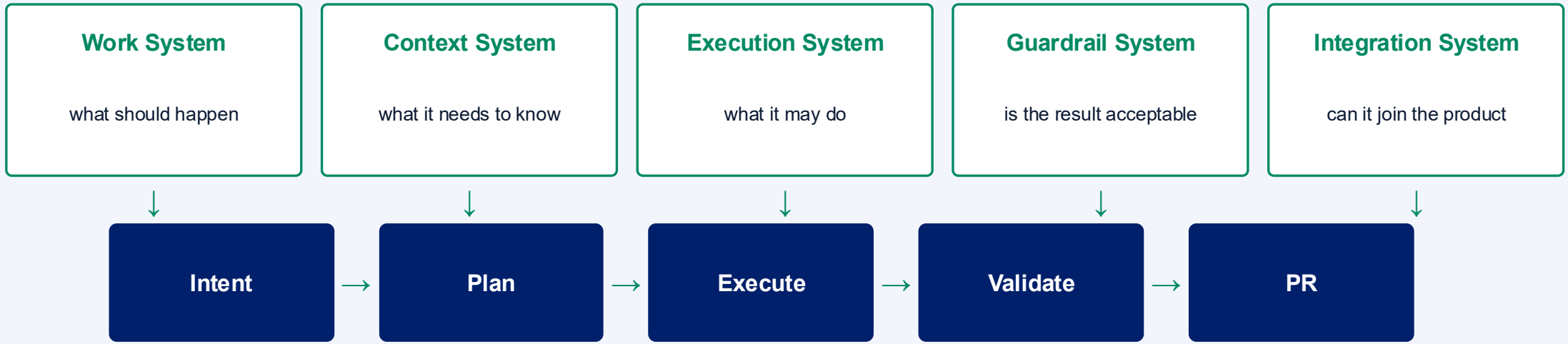
NOW CONSTRAINED

- Requirements & work-intake quality
- Architecture decisions
- Feature decomposition
- Product decision throughput
- PR review capacity
- Security & legal review
- Documentation quality (not volume)
- Support readiness for change

Local developer productivity is not system throughput. Optimize the constraint, not the keyboard.

REFERENCE ARCHITECTURE

The Agent Is Only One Component of an Agentic Engineering System



Every pattern in the rest of this talk plugs into one of these five systems.

The agent gets freedom inside the engineered system. It does not get freedom to redefine the system.

WORK SYSTEM

Agentic Development Starts With Better Work Decomposition

WORK HIERARCHY

Epic / Master Feature

larger area of value

Feature

smallest usable business outcome

Story

demoable sub-plan tied to AC + NFRs

Agent Tasks

bounded execution steps

SOURCE-CONTROL MAPPING

Task	→	**Commit**
Story	→	**PR into feature branch**
Feature	→	**Integration PR**

Feature flags decouple integration from release.

Structured decomposition is what makes parallel agent execution traceable instead of chaotic.

REUSABLE TEMPLATE

Before an Agent Starts, the Work Must Be Agent-Ready

Intent

What business or user outcome should exist

Scope

Where change is allowed

Out of Scope

Explicit boundaries agents must not expand

Acceptance Criteria

Observable, testable success

Non-Functional Requirements

Security, performance, compatibility, operability

Architecture Constraints

Relevant ADRs, boundaries, patterns

Allowed Operations

Which commands / tools may be executed

Validation

How correctness will be demonstrated

Definition of Complete

All requirements met, no unrelated behavior change, guardrails pass

A ticket existing does not mean it is ready for autonomous execution.

WORK SYSTEM

If Code Is Cheap, Requirements Engineering Becomes More Valuable

READINESS EVALUATION — WHAT THE PLANNING AGENT CHECKS

- Ambiguity
- Missing acceptance criteria
- Missing NFRs
- Affected architecture
- Dependencies
- Testability
- Scope creep risk
- Delivery / change risk

Product does not need to become a prompt-writing organization. It needs to engineer clearer requirements.

EXECUTION SYSTEM

Do Not Assume One Model or One Context Should Do Everything

PLAN

- Higher-reasoning model
- Broad context
- Architecture & tradeoffs
- Decomposition into tasks

→

EXECUTE

- Coding / task-specific model
- Narrow context
- Bounded tools & commands
- Iterative implementation

→

VALIDATE

- Fresh context
- Original requirements
- Deterministic checks
- Independent evaluation

Swappable by work type: planning model | coding model | documentation model | coverage analysis | security analysis

Not every plan deserves the same model, and not every task needs the same model.

CONTEXT SYSTEM

Context Should Be Staged — Not Accumulated

ALWAYS-ON	▪ Critical repository instructions ▪ Global constraints ▪ How to discover more context
CONDITIONAL	▪ Subsystem instructions ▪ Language & testing rules ▪ Relevant ADRs
DISCOVERABLE	▪ Documentation & architecture catalogue ▪ Backstage, source code ▪ RAG / MCP where appropriate

SEPARATE CONTEXT WINDOWS

Planning Context

≠

Implementation Context

≠

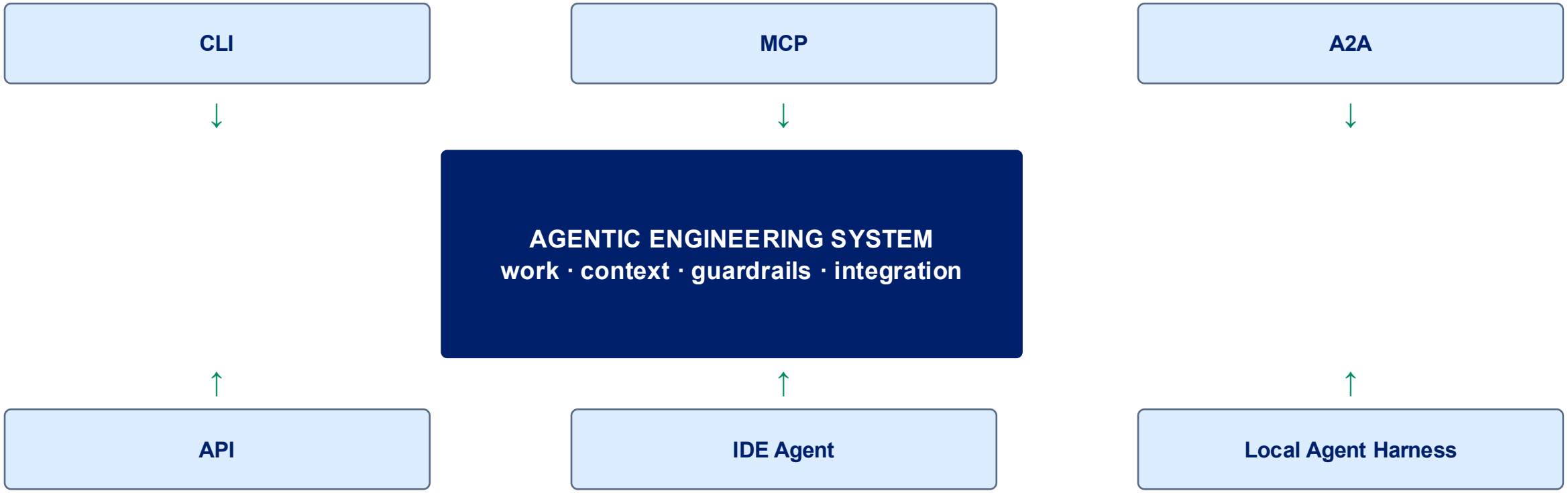
Validation Context

Long conversations are repeatedly summarized — fidelity is lost, and implementation history quietly becomes the requirement.

Fresh context can act as independent review. Validation should return to the original requirements.

INTERFACES

Build the Operating Model So It Does Not Depend on Which Agent Protocol Wins



- MCP makes capabilities discoverable, but static MCP context consumes window capacity · CLI interfaces consume less default context but need tighter execution controls · Protocol choice is secondary to engineering-system design.

EXECUTION SYSTEM

Agents Should Use Approved Interfaces — Not Arbitrary Shell Commands

SANDBOXED AGENT

Agent

Sandbox options (choose per requirement):

- devcontainer
- worktree
- OpenShell
- firejail
- disposable container / VM

ALLOWED INTERFACE

```
task build
task test
task lint
task validate
task security
```

Same commands for humans, agents, local dev, and CI.

BLOCKED

- Arbitrary shell
- Direct CI modification
- Deployment
- Unrestricted network
- Uncontrolled dependency installs

The interface is the contract between the agent and the engineering environment.

GUARDRAIL SYSTEM

Guardrails Exist at Four Different Layers

1 EXECUTION

How can the agent act?

- sandbox
- permissions
- approved commands
- network
- runtime limits

2 ENGINEERING

Is the implementation acceptable?

- tests
- lint
- security
- dependencies
- architecture
- API compatibility
- NFRs

3 PROCESS

Does the agent have authority?

- plan review
- task size
- autonomy level
- branch policy
- human checkpoints

4 INTEGRATION

Can this safely join the product?

- regression
- integration
- packaging
- supply chain
- feature flags
- CI/CD policy

Passing local validation does not automatically mean the product is releasable.

AUTONOMY POLICY

Autonomy Should Be Risk-Based, Not Model-Based

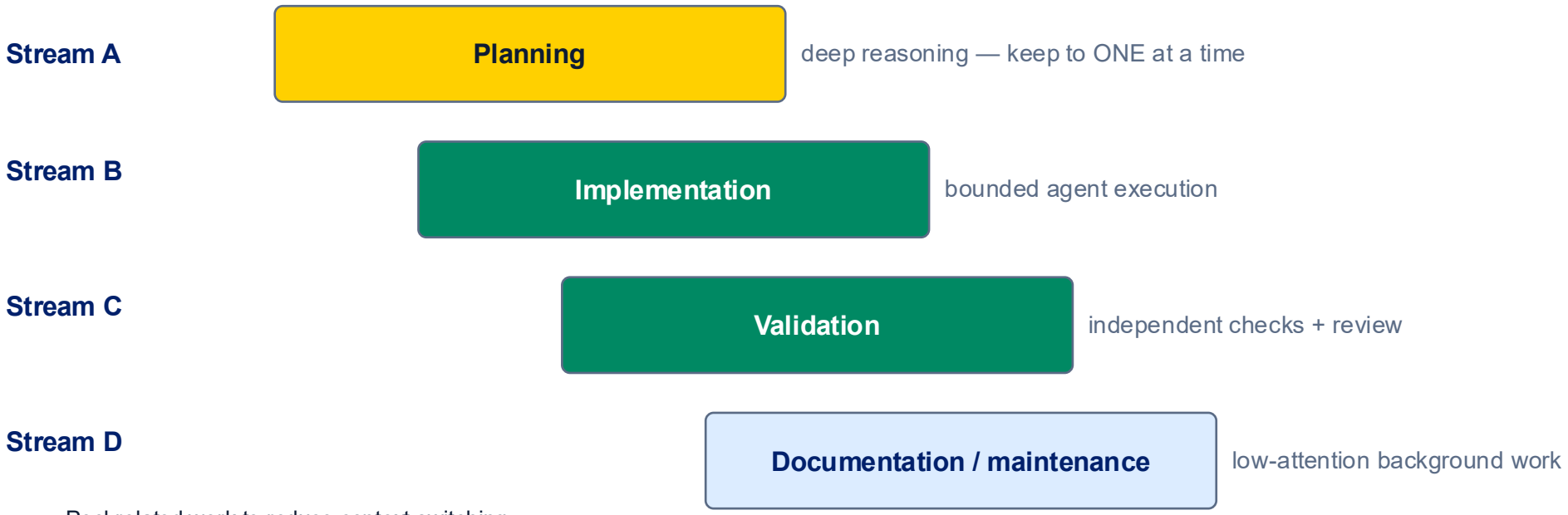
Work Type	Plan Review	Agent Execution	Validation	Human PR Review
Dependency Update	Usually none	Autonomous	Strict automated checks	Yes
Documentation	Usually none	Autonomous	Link / style / content checks	Lightweight
Bug Fix	Sometimes	Autonomous	Tests + regression	Yes
Small Feature	Required	Autonomous after plan	Full engineering checks	Yes
Architecture Change	Mandatory	Highly bounded	Expanded validation	Senior review

A dependency update and a new architecture should not have the same autonomy policy.

OPERATING PRACTICE

Agents Increase Parallel Capacity — Human Cognition Becomes the WIP Limit

One engineer supervising staggered streams



- Pool related work to reduce context switching
- Stagger task maturity so only one stream needs deep reasoning
- Engineers increasingly work in review and integration cycles
- Not every engineer thrives at the same parallelism level

Parallelism is real, but bounded by attention — not by how many agents you can launch.

MEASUREMENT

Traditional Output Metrics Become Less Useful When Code Is Cheap

STOP OPTIMIZING FOR

- Lines of code
- Raw code generation volume
- Raw commit count
- Generated documentation volume

MEASURE THE SYSTEM

- Time to plan
- Time to first review
- PR cycle time
- Task acceptance time
- Feature lead time
- Human interventions
- Escaped defects
- Parallel task capacity
- Token consumption vs. delivered outcome

Nuance: PR count still helps when PRs deliberately map to meaningful work checkpoints.

Raw output is now trivially manufactured. Cycle time and intervention rate tell you the truth.

MATURITY MODEL

Do Not Jump From Autocomplete Directly to Autonomy

increasing autonomy →

CRAWL

AI helps the developer

Small, low-risk, reversible tasks. Individual adoption.

WALK

AI collaborates with the developer

Multi-file work, tests, documentation, reviews.

RUN

Engineers delegate bounded work

Agents become part of normal delivery workflows.

SPRINT

Agentic workflows embedded across the SDLC

Preparation, execution, validation, integration orchestrated.

Increase autonomy only as the engineering system earns trust.

PROOF POINT · BEFORE

Seven Years of Engineering Effort Still Could Not Keep Pace

Problem domain: a hand-built extension / resource-pack ecosystem tracking fast-moving external platform APIs.

~7 years

of accumulated engineering effort

2–5 engineers

dedicated specialist team

APIs outran us

external platforms evolved faster than we could implement

Brittle generation

earlier code-gen attempts stayed specialist-heavy

Coverage shrank

support for several platforms reduced or abandoned

When producing code is treated as bespoke craftsmanship, more effort never catches up to the change rate.

PROOF POINT · AFTER

The Breakthrough Was Not Faster Typing — It Was Making the Work Repeatable

BEFORE

- Years of accumulated work
- Specialist-only team
- Shrinking platform coverage
- Brittle, bespoke generation

ENGINEERED AGENTIC WORKFLOW

New pack	**~45 min**	Single resource	**< 5 min**
Service area	**~1 hour**	Full end-user deliverable	**< 1 week**

Outputs are not just source code:

implementation	**tests**	**documentation**	**simulator**
knowledge explorer	**CI automation**	**drift detection / auto-PR**	**coverage validation**

Systematization created the leverage. Agents made the system economically viable.

LIVE DEMO · ~12 MINUTES

Demo: A Complete Agentic Engineering Loop

failure fe

WHAT IS RUNNING

```
OpenCode (local agent harness)
qwen3.6:35b-a3b-bf16 (local model)
sandboxed / bounded environment
repository instruction files
task abstraction (build/test/lint)
deterministic policy + test checks
```

Intentionally a modest local model — the point is the system around it, not the model.

LIVE DEMO · ~12 MINUTES

Demo: A Complete Agentic Engineering Loop

AFTER THE DEMO

A Good Engineering System Is Designed to Catch Imperfect Implementation

Mistakes are expected. What matters is that they occur inside a bounded environment and are caught by deterministic controls before they become integrated product behavior.

Do not design a system that requires the model to be right on the first try.

MONDAY MORNING

Start With One Engineered Agentic Workflow

1. Pick one bounded, repeatable engineering task.
2. Define an approved execution interface (task / make).
3. Isolate the execution environment.
4. Encode 3–5 critical engineering rules.
5. Define deterministic acceptance checks.
6. Separate planning, execution, and validation context.
7. Require the agent to pass the same checks as humans.
8. Measure cycle time and human intervention.
9. Increase autonomy only after the workflow proves reliable.

Do not begin by transforming the entire SDLC. Engineer one repeatable workflow, then expand.

Agentic Development Is an Engineering-System Problem

1

Stop treating AI as magic. Treat it as the next abstraction layer.

2

Code is becoming cheap. Engineering judgment is becoming the constraint.

3

Probabilistic workers require deterministic engineering systems.

The goal is not autonomous code generation.
The goal is reliable business outcomes from abundant implementation capacity.

Questions · Appendix slides follow

DEFINITIONS

Appendix A — A Spectrum, Not Three Products

AI-Assisted Development

Human drives every step. AI suggests, completes, and explains. All execution decisions remain human.

Agentic Development

Human defines intent, constraints, and validation. The agent plans and executes bounded work inside an engineered system.

Autonomous Engineering

Workflows execute end-to-end with human oversight at risk-based checkpoints only. Rare, and earned.

`human-driven → increasingly delegated execution`

REUSABLE EXAMPLE

Appendix B — Example Agent-Ready Story

Intent	Users can filter the audit-log API by date range so support can investigate incidents without exporting full logs.
Scope	services/audit-api query layer, request validation, OpenAPI spec, unit + integration tests.
Out of Scope	Storage schema changes, UI, retention policy, authn/authz behavior, other endpoints.
Acceptance Criteria	from/to ISO-8601 params; inclusive range; 400 on invalid or inverted range; default unchanged when omitted; paging preserved.
NFRs	p95 < 300ms at 10M rows; no new dependency; backward compatible; structured logs on rejection.
Architecture Constraints	ADR-014 query-builder boundary; no raw SQL in handlers; repository interface unchanged.
Allowed Operations	task build, task test, task lint, task validate. No network, no dependency installs, no CI edits.
Validation	New unit tests + integration test against the seeded fixture; contract test vs. published OpenAPI; existing suite green.
Definition of Complete	All AC + NFRs met, no unrelated behavior change, all four guardrail layers pass, PR opened against the feature branch.

REPOSITORY STRUCTURE

Appendix C — Architecture, Instructions, Tasks, and Policies Are First-Class Artifacts

```
/docs
    /adr                architecture decision records
    /architecture       catalogue, boundaries, diagrams

/agent
    instructions.md     always-on repository rules
    /conditional        subsystem & language guidance

/tasks
    Taskfile.yml        approved execution interface

/src
/tests
/policies               dependency, license, security
/.devcontainer          sandbox definition
```

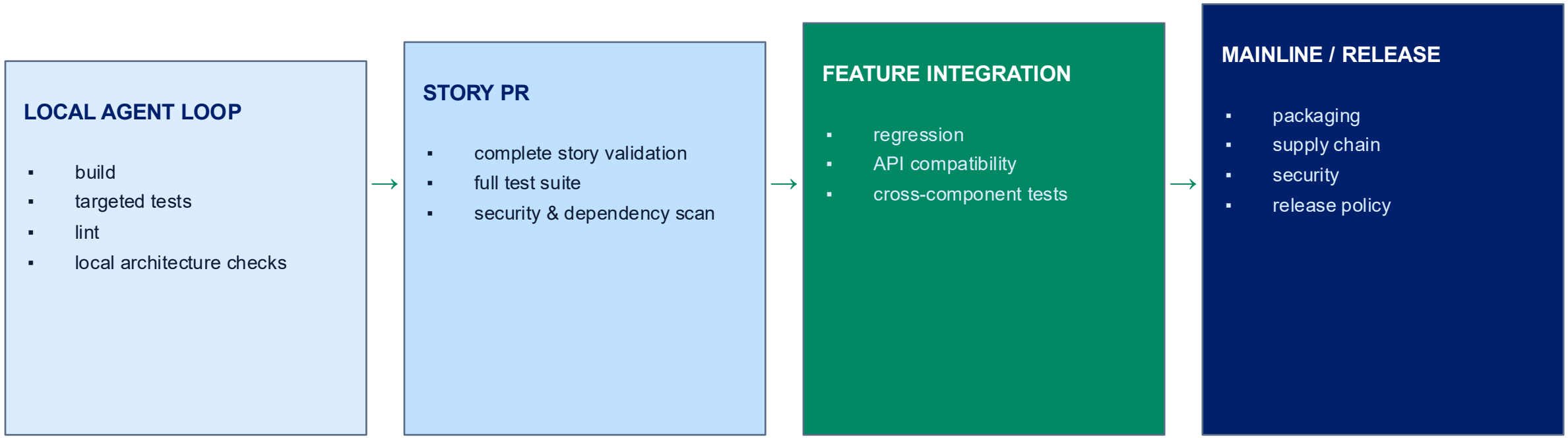
WHY IT MATTERS

- Instructions live with the code they govern and version with it
- Executable tasks give humans, agents, and CI one interface
- Policies are enforceable, not aspirational
- ADRs give agents the 'why' behind boundaries
- Sandbox config makes bounded execution reproducible

Exact filenames are illustrative — the structure is the point.

GUARDRAIL PIPELINE

Appendix D — Guardrails Broaden as Work Moves Toward Release



Scope of validation widens with blast radius — fast and narrow locally, broad and slow at release.

RISK POLICY

Appendix E — Risk Dimensions Determine Autonomy, Review, and Validation Depth

RISK DIMENSIONS (SCORE EACH)

- Reversibility — can it be rolled back cheaply?
- Customer impact — who notices if it is wrong?
- Architecture impact — does it move a boundary?
- Security impact — authn, authz, secrets, data paths
- API impact — published contract or compatibility
- Data impact — migration, retention, correctness
- Validation quality — how good are the existing checks?

→

WHAT THE SCORE SETS

Required planning review

`none → peer → senior → architecture board`

Permitted agent authority

`autonomous → bounded → plan-gated → assisted only`

Validation depth

`targeted tests → full suite → regression + compatibility`

Human review level

`lightweight → standard → senior + second reviewer`

ANTI-PATTERNS

Appendix F — What Not to Do With Agent Context

- ✕ One enormous AGENTS.md that everything is appended to
- ✕ Putting every document into every prompt
- ✕ Using one conversation from planning through validation
- ✕ Relying on repeated summarization indefinitely
- ✕ Letting implementation history redefine the requirement
- ✕ Repeatedly fetching static knowledge that should be durable

Context is a budget. Spend it deliberately.